

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- EXPERIMENTS

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO3	Assessment:	ASSESSMENT TOOL 1- EXPERIMENTS
Weightage:	40%	Total Marks:	25
CO3 Statement:	<i>Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)</i>		
Student Name:	SREYA.B	Reg.No.:	192521442
Department:	IT	Date:	30/07/26

ANNEXURE A

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.

ANS:

The proposed system is designed to perform register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. The system uses multiple 8-bit registers, such as R0, R1, R2, and R3, where R1 and R2 hold the input operands and R3 stores the result. Register Transfer Language (RTL) is used to represent the movement of data between registers, such as $R1 \leftarrow R0$. The Arithmetic Logic Unit (ALU) receives data from the registers and performs the selected arithmetic or logical operation based on the control signals generated by the control unit. For example, when the operation $R3 \leftarrow R1 + R2$ is executed, the values stored in R1 and R2 are transferred to the ALU, the ALU performs the addition, and the resulting value is stored in R3. Similarly, subtraction, AND, and OR operations can be performed using the same datapath. For example, if $R1 = 20$ and $R2 = 5$, the ALU produces 25 for addition, 15 for subtraction, 4 for AND, and 21 for OR. The sequence of execution involves selecting the source registers, transferring operands to the ALU, selecting the required operation through control signals, processing the data in the ALU, and transferring the result to the destination register. Thus, the system demonstrates how registers act as temporary storage locations, how the ALU performs data processing, and how the control unit coordinates the overall operation. The design can be simulated using Verilog to observe register values and verify the correctness of each operation. This system provides a basic understanding of the datapath and the internal working of a processor.

2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.

ANS:

The proposed computer system model demonstrates data transfer using both single-bus and multi-bus organizations. In the single-bus organization, all registers, memory, and I/O devices share one common bus for communication, so only one data transfer can take place at a time. For example, data can be transferred from a register to memory or from an I/O device to a register through the same bus. The control unit selects the required source and destination and controls the transfer sequentially. Although this design is simple and requires less hardware, it may cause bus contention and delays when multiple components need to communicate simultaneously. In a multi-bus organization, two or more buses are provided, allowing multiple data transfers to occur at the same time. For example, one bus can transfer data between registers while another bus communicates with memory or an I/O device. This parallel data transfer reduces waiting time and improves overall system throughput. When the same data transfer operations are performed using both approaches, the single-bus system generally requires more clock cycles because transfers must occur sequentially, whereas the multi-bus system can complete some operations simultaneously. Therefore, the multi-bus organization provides better efficiency, lower transfer time, and reduced bottlenecks, although it requires additional hardware and more complex control logic. The model clearly shows that multiple bus structures improve processor performance by enabling parallel communication between registers, memory, and I/O devices.

3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.

ANS:

A 5-stage instruction pipeline is designed and simulated using the stages Fetch (IF), Decode (ID), Execute (EX), Memory Access (MEM), and Write-back (WB). A set of instructions is passed through these stages so that different instructions can be processed simultaneously at different stages. In a non-pipelined processor, each instruction must complete all five stages before the next instruction starts, resulting in a total execution time of approximately $5 \times N$ clock cycles for N instructions. In the pipelined system, after the pipeline is filled, one instruction can complete every clock cycle, so the execution time is approximately $N + 4$ clock cycles. For example, for 6 instructions, the non-pipelined system requires 30 clock cycles, while the pipelined system requires only 10 clock cycles. The speedup can be calculated as the ratio of non-pipelined execution time to pipelined execution time, giving $30/10 = 3$. The throughput of the pipeline is also improved because multiple instructions are processed simultaneously, and after the initial four-cycle filling period, one instruction can be completed per

clock cycle. The simulation demonstrates that pipelining significantly improves processor performance by increasing instruction throughput and reducing the average execution time per instruction. However, practical performance may be affected by hazards such as data, control, and structural hazards, which can introduce stalls and reduce the ideal speedup. Thus, the 5-stage pipeline provides a clear improvement over a non-pipelined architecture by allowing instruction-level parallelism and better utilization of processor resources.

4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.

ANS:

A pipeline execution scenario is developed using a 5-stage instruction pipeline consisting of Fetch (IF), Decode (ID), Execute (EX), Memory (MEM), and Write-back (WB) stages to demonstrate data, control, and structural hazards. A data hazard occurs when one instruction depends on the result of a previous instruction, such as ADD R1, R2, R3 followed by SUB R4, R1, R5. Since the second instruction requires the updated value of R1, it may have to wait. This problem can be reduced using data forwarding, where the result from the Execute or Memory stage is directly forwarded to the next instruction without waiting for the Write-back stage. A control hazard occurs when a branch instruction changes the flow of execution, for example, BEQ R1, R2, LABEL. The processor may fetch the wrong instruction before the branch decision is known. This can be handled using branch prediction, where the processor predicts whether the branch will be taken and continues fetching instructions based on the prediction. If the prediction is incorrect, the incorrectly fetched instructions are flushed and the correct instructions are loaded. A structural hazard occurs when two instructions require the same hardware resource at the same time, such as a single memory unit being accessed simultaneously for instruction fetch and data access. This can be resolved by inserting stalls or providing separate instruction and data memories. During simulation, the pipeline is observed before and after applying these techniques. Without hazard handling, instructions may produce incorrect results or require several idle clock cycles. With stalling, forwarding, and branch prediction, the number of pipeline stalls and wasted cycles is reduced, improving execution efficiency and throughput. Therefore, the simulation demonstrates that effective hazard detection and resolution techniques are essential for maintaining correct instruction execution while achieving the performance benefits of pipelining.

5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

ANS:

A processor supporting superscalar execution is designed to execute multiple instructions simultaneously by using more than one execution unit. In a superscalar processor, instructions are fetched and decoded together, and independent instructions can be issued to different functional units such as the ALU, floating-point unit, or load/store unit during the same clock cycle. Out-of-order execution improves performance by allowing an instruction that is ready to execute to proceed

even when an earlier instruction is waiting for data or another resource, while maintaining the correct final program result. Speculative execution further improves performance by predicting the outcome of branches and executing instructions before the actual branch result is known; incorrect speculative instructions are discarded. Hyper-threading and simultaneous multithreading (SMT) extend this concept by allowing multiple hardware threads to share the processor's execution resources. For example, when one thread is waiting for memory access, instructions from another thread can use otherwise idle execution units, increasing resource utilization. In the simulation, independent instructions from one or more threads are scheduled across multiple execution units, and the number of instructions completed per cycle is measured. A single-issue processor may execute approximately one instruction per cycle under ideal conditions, whereas a four-issue superscalar processor can potentially execute up to four independent instructions per cycle. With out-of-order and speculative execution, more execution units remain active and pipeline stalls are reduced. With SMT, additional threads can improve utilization when a single thread cannot fully use the available resources. Therefore, superscalar execution, out-of-order scheduling, speculative execution, hyper-threading, and SMT can significantly increase instruction throughput and resource utilization compared with a conventional single-issue processor. However, the actual improvement depends on instruction-level parallelism, branch prediction accuracy, memory delays, dependencies, and contention for shared resources.

Code of Conduct:

*I **SREYA.B(192521442)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

SREYA.B

MARKS ALLOCATION

	Understanding of Concepts (20%)	Experimental Design (20%)	Use of Tools (25%)	Analysis of Results (20%)	Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation